

```
| x < 0 = "Negative"
| otherwise = "Zero"
```

The guards have to be neatly aligned:

```
*Main> sign 0
"Zero"
*Main> sign 3
"Positive"
*Main> sign (-3)
"Negative"
```

There are three very important higher order functions in Haskell — *map*, *filter* and *fold*.

The *map* function takes a function and a list, and applies the function to each and every element of the list. Its type signature is:

```
*Main> :t map
map :: (a -> b) -> [a] -> [b]
```

The first function argument accepts an element of type 'a' and returns an element of type 'b'. An example of adding two to every element in a list can be implemented using *map*:

```
*Main> map (+ 2) [1, 2, 3, 4, 5]
[3,4,5,6,7]
```

The *filter* function accepts a *predicate* function for evaluation, and a list, and returns the list with those elements that satisfy the *predicate*. For example:

```
*Main> filter (> 0) [-2, -1, 0, 1, 2]
[1,2]
```

Its type signature is:

```
filter :: (a -> Bool) -> [a] -> [a]
```

The *predicate* function for filter takes as its first argument an element of type 'a' and returns *True* or *False*.

The *fold* function performs a cumulative operation on a list. It takes as arguments a function, an accumulator (starting with an initial value) and a list. It cumulatively aggregates the computation of the function on the accumulator value as well as each member of the list. There are two types of *folds* — *left* and *right fold*.

```
*Main> foldl (+) 0 [1, 2, 3, 4, 5]
15
*Main> foldr (+) 0 [1, 2, 3, 4, 5]
15
```

Their type signatures are, respectively:

```
*Main> :t foldl
foldl :: (a -> b -> a) -> a -> [b] -> a
*Main> :t foldr
foldr :: (a -> b -> b) -> b -> [a] -> b
```

The way the *fold* is evaluated among the two types is different and is demonstrated below:

```
*Main> foldl (+) 0 [1, 2, 3]
6
*Main> foldl (+) 1 [2, 3]
6
*Main> foldl (+) 3 [3]
6
```

It can be represented as '*f (f (f a b1) b2) b3*' where '*f*' is the function, '*a*' is the accumulator value, and '*b1*', '*b2*' and '*b3*' are the elements of the list. The parenthesis is accumulated on the left for a *left fold*. The computation looks like this:

```
*Main> (+) ((+) ((+) 0 1) 2) 3
6
*Main> (+) 0 1
1
*Main> (+) ((+) 0 1) 2
3
*Main> (+) ((+) ((+) 0 1) 2) 3
6
```

With the recursion, the expression is constructed and evaluated only when it is finally formed. It can thus cause stack overflow or never complete when working with infinite lists. The *foldr* evaluation looks like this:

```
*Main> foldr (+) 0 [1, 2, 3]
6
*Main> foldr (+) 0 [1, 2] + 3
6
*Main> foldr (+) 0 [1] + 2 + 3
6
```

It can be represented as '*f b1' (f b2 (f b3 a))*' where *f* is the function, '*a*' is the accumulator value, and '*b1*', '*b2*' and '*b3*' are the elements of the list. The computation looks like this:

```
*Main> (+) 1 ((+) 2 ((+) 3 0))
6
*Main> (+) 3 0
3
*Main> (+) 2 ((+) 3 0)
5
*Main> (+) 1 ((+) 2 ((+) 3 0))
6
```

To be continued on page.... 44